XH-202625
基于不均衡小样本学习的光学遥感卫星陆上目标检测识别比赛方案

Our project:
A system for object detection on satellite imagery under low-data conditions

Tasks before the project:
Development of an algorithm capable of detecting rare ground objects in optical satellite imagery—even when there are very few examples of such objects and they occur far less frequently than others.

Issues before the project:
A new satellite has been launched. We need to quickly train it to "see" the required objects on the ground. However, there are only a handful of labeled images, and some objects are abundant while others are nearly absent. Conventional algorithms perform poorly under these conditions: they either miss rare objects or generate false alarms.

Solution
A three-stage training system—first on large open datasets, then fine-tuning on our own data, plus specialized techniques for handling class imbalance and few-shot scenarios.

What is the essence of the task?

Our team need to teach the computer to automatically find objects of interest in these photos — for example, military equipment, temporary buildings, ships — and frame them.
It sounds simple, but there are three big problems:

- Problem 1 — there are few examples. Normal models learn from millions of photos with captions. We have only a few or dozens of images with the right objects. This is called a few-shot (literally "few shots").
- Problem 2 is the skew in classes. Imagine that you have 500 pictures with airplanes, 50 with cars and only 5 with special equipment. If you just train the model on this, it will be good at finding planes and almost ignore special equipment. This is called unbalanced (unbalanced data).
- Problem 3 is the specificity of satellite images. The objects are small, looking from above, shadows, clouds, different lighting. Algorithms trained on ordinary photos (cat, dog, car on the side) work poorly here.

What they estimate: we need to find Recall ≥ 85% of the objects (not to miss) and FAR ≤ 20% (no more than 20% of false alarms), and all this in 20 seconds for one huge 10,000×10,000 pixel picture.

Project architecture

Step 1 - Data Preparation

How we handle the lack of training examples

Image slicing
Satellite images are huge (10k×10k px).
We cut them into small 1024×1024pieces with overlap so nothing is missed

Data multiplying
We cut out rare objects and paste them into different images. Turns 5 examples into hundreds instantly

Scene mixing
We combine 4–9 images into one training sample.
Model sees more variety per step and learns faster

Fair training
Rare classes are shown more often. Model doesn't ignore them because they're uncommon

Step 2 - Detection Model

How the model looks at an image and finds objects

Eyes (Backbone)
Looks at the image and extracts basic features: edges, shapes, textures.
Pre-trained on millions of images

Scale adapter
Objects vary a lot in size from above. This part combines features at 5 scales so big and small objects are both seen

Brain (Decoder)
Uses attention to focus on important areas. Decides "something is here« without needing extra filter steps

Output head
Draws the box around object and names its class.
Penalizes missed rare objects more

Step 3 - Training Strategy

We teach the model in three stages — from general to specific

Stage 1 — Learn basics
Train on large public satellite datasets(thousands of images).Model learns what objects look like
from above in general.
Backbone is frozen —only the detector
layers are updated.
Goal: strong starting point for next stages

Stage 2 — Specialize
Now train on the competition dataset from the organizer.
Unlock upper layers so model adapts to our specific targets.
Use a smarter loss that punishes harder for missing rare ones.
Heavy augmentation: copy-paste, mosaic, flips, color shifts.
Lower learning rate

Step 4 - Inference

(≤20 sec / 10k×10k image, single GPU)
How the final model processes a full satellite image at competition time

Slice image
The 10k×10k image is cut into 1024px tiles with overlap.
8 tiles processed in parallel per GPU batch

Run detector
Each tile goes through the model.
Optimized for speed using FP16 precision — twice as fast, same accuracy

Merge results
Detections from all tiles are combined. Duplicate boxes on tile borders are merged smartly using weighted avg

Final output
Results saved as standard JSON file with all bounding boxes + classes. + visual images with boxes drawn

Detailed description of the architecture of the project:

1) Step 1 - Data Preparation

The first problem is that we don't have enough images of rare objects.

We bypass it like this:

- **Slicing into pieces.** A 10,000×10,000 picture is huge. The neural network cannot process it entirely. We cut it into 1024×1024 pixel pieces with a small overlap (so that the object doesn't get lost on the border of the pieces), process each one, and then glue the results back together.
- **Copy-Paste augmentation.** We take a rare object from a picture, cut it out, and paste it into different places in other pictures. This is how we artificially create thousands of new examples from dozens of real ones. It's like a copier for rare objects.
- **Load balancer.** When teaching, we specifically show the model rare classes more often than frequent ones, so that she doesn't "get hung up" on what is most important.

2) Step 2 — Model Architecture

The model consists of three parts:

- The backbone is the "eyes" of the model. She looks at the picture and extracts the signs: where are the edges, where are the textures, where are the shapes. We use ConvNeXt, a network that has already been pre—trained on millions of ordinary photos, and then retrained on satellite images. It's like hiring someone with experience and then retrain them to suit your needs.
- Objects in satellite images come in very different sizes: an airplane can occupy 200 pixels, and a car 20. Neck combines information from different scales so that the model sees both large and small objects equally well. Technically, this is the FPN (Feature Pyramid Network), a pyramid of features.
- Brain - makes the final decision: "here is the object, here is its class, here are the coordinates of the frame." We use RT-DETR, a modern transformer—based detector without an unnecessary filtering stage (NMS), which saves time.

Detailed description of the architecture of the project:

1) Step 3 - Training Strategy

We train in two stages (the third stage is optional in terms of time and skills /strengths of the team):

- Stage 1 — Pre-training on a large dataset. First, we take open datasets of satellite images (DOTA, DIOR) with thousands of marked objects. We teach the model to understand «what is an object in a satellite image in general».
- Stage 2 is Fine-tuning on our data. We take the learned model and train it on the data of our specific competition. We use a special GCL loss function, which is smarter than the standard one — it penalizes the model more strongly for errors in rare classes.
- *Stage 3 is Meta-learning. Additionally, we train the model to be flexible: we show her some examples of a new class and ask her to recognize it in other pictures. It's like teaching a person to learn — not to memorize specific objects, but to understand the principle of «what a new thing looks like.»*

2) Step 4 —Inference

When the model is ready and we were given a test picture:

- We cut the 10000×10000 picture into pieces
- We run each piece through the model (in parallel, in batches of 8),
- Glue the results back together — we remove duplicate frames through WBF
- We issue COCO JSON with the coordinates of all found objects.
- For speed, the model is converted to TensorRT

Separation of tasks:

Field	Who	Skills	Tasks
ML	Has implemented model training in PyTorch	Can launch training, track metrics, modify architecture, and understand existing model code (including with AI-assistant)	Take the existing architecture, fine-tune it on our data, tune hyperparameters, and compare results
Data	Has worked with NumPy/Pandas	Able to write scripts to process image folders, analyze class distributions, crop images into patches, and visualize annotations, make simple EDA	Crop images into tiles, augment rare classes via copy-paste, and verify annotation accuracy